第 2 章　Java对象设计概述

教学目标：

概括性地掌握java语言的主要特性，能编写简单的java应用程序。

2.1 引子

- Java程序由类构成，从一个类可以创建任意多个对象。
- 类的成分：域和方法。
- 类实质上是描述了由它所创建的每个对象所具有的状态和方法。

- 例

主类

作为程序执行入口的主方法

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello,World");
    }
}
```

与C++不同：
java类中的成员方法的定义必须在类体内完成

- 这段源程序无论如何命名（扩展名必须是java），编译后的字节码文件必定为HelloWorld.class，如果该类为public的，则源文件名也必须是HelloWorld。
- 字节码是运行在JVM上的机器语言。

- 执行结果是输出字符串Hello,World。
- 作为java application，必须有一个类含有main方法，这是程序执行的入口。
- 作为一个最简单的application，没有域，只有一个main方法，且其中只有一条语句。
- System是系统定义的一个类，out是它的一个静态域， println是out的一个方法。

2.2 变量

- 内置的基本数据类型:
  - 整型:
    - byte 8位整数（带符号）
    - short 16位整数（带符号）
    - int 32位整数（带符号）
    - long 64位整数（带符号）
  - 浮点型:
    - float 32位浮点数
    - double 64位浮点数

– 布尔型：

- boolean true/false （不同于1/0）

– 字符型：

- 16位unicode字符

- 引用类型：类、接口和数组。
- 程序员定义的对象类型需事先定义。
- java没有缺省类型，变量的类型必须显式定义。
- 变量在初试化之前没有定义，不能使用未赋值的变量。

- 例

```
public class Test {
    public static void main(String[] args) {
        int i;
        int j=i+1;
    }
}
```

命令提示符

```
D:\lecture\java\2004\example\chap0>javac Test.java
Test.java:4: variable i might not have been initialized
                int j=i+1;
                      ^
1 error
```

- 例

```
public class PartialSum {
    public static void main(String[] args) {
        int s=0;
        int i=1;
        while(i<=50) {
            s=s+i*i;
            i=i+1;
            System.out.println(s);
        }
    }
}
```

初始化

- println是一种名复用方法，可以接受不同类型参数

2.3 注释

- 整行注释：//
- 整段注释：/*和*/之间
- 文本注释：/**和*/之间，一般用于描述其后的内容，javadoc工具可以用来提取文本注释，形成html文档
- 注释不能嵌套，例

```
/*  ……
   /*……************…….
*/
```

2.4 命名常量

- 常量：程序执行过程中保持不变。
- 使用有名常量的好处：
  - 名字能描述量值的含义；
  - 在一处定义，便于维护。
- Java中，创建有名常量是用static和final来说明一个变量，并在说明中提供一个初始化值，例

```
public class CircleStuff {
    public static final double π=3.1416;
}
```

- 可以把相关的常量组织起来放在一个类里，例

```
public class Suit {
    public final static int CLUBS=1;
    public final static int DIAMONDS=2;
    public final static int HEARTS=3;
    public final static int SPADES=4;
}
```

使用Suit.CLUBS，Suit.HEARTS来访问

2.5 控制流

- 控制流语句描述的是在程序运行时决定执行哪条语句的问题，主要有：while,if/else,for,switch,do/while,还有“块”，块是用{}封状起来的多个语句，也被称为“分程序”。
- 例: 改进的PartialSum程序，输出结果中标注了序列号，并用*标注所有偶数。

```
public class ImprovedPartialSum {
  static final int MAX_INDEX=10;
   public static void main(String[] args) {
     int s=0;
     String mark;
     for (int i=1;i<MAX_INDEX;i++) {
        s=s+i*i;
        if (s%2==0) mark="*";
        else mark= "";
              System.out.println(i+":"+s+mark);
     }
  }
}
```

是Java的一个类

指向

转换为字符串对象

运行结果：

1:1

2:5

3:14*

4:30*

5:55

6:91

…

- 其中输出语句使用+把表示i的字符串、一个分隔符、表示s的字符串和一个标志字符串连接在一起。对于字符串，+是连接操作，对于算术表达式则是加法运算，例如

```
int i=3,j=2;
System.out.println(i+"*"+j+"="+(i*j));//3*2=6
System.out.println(i+"+"+j+"="+i+j);//3+2=32
System.out.println(i+"-"+j+"="+i-j);//编译报错
```

先

减法不支持转换字符串

2.6 类和对象

- 每个对象都有一种类型，就是对象所属的类。
- 每个类有两种主要成员：域和方法。
- public class Point {

```
    public double x,y;
}
```

2.6.1 创建对象

- 使用关键词new。
- 从类创建对象的过程称为实例化，对象也称为实例
- 新创建的对象放在被称为“堆”的系统存储区中。
- 对象的访问通过“对象引用”(object reference)：持有一个对象的变量实质上就表示对该对象的引用。当变量不再引用任何对象时，该对象引用就是null。
- 引用是有类型的。
- 把对象传递给一个方法实际所做的是把对象的引用传递给一个方法。

- 例

```
Point lowerLeft=new Point();
Point upperRight=new Point();
Point middlePoint=new Point();
lowerLeft.x=0.0; lowerLeft.y=0.0;
upperRight.x=1280.0; upperRight.y=1024.0;
middlePoint.x=640.0; middlePoint.y=512.0;
```

创建三个独立的Point对象，分别定义三个引用指向它们。

- 每个Point对象都有独立的x,y
- 对象中的x,y域称为实例变量

- 例

```
public class Demo {
  public static void main(String[] arg
    Point p1,p2;
    new Point();
    p1=new Point(); p2=p1;
    p1.x=1; p1.y=2;
    System.out.println(p2.x+", "+p2.y);
}}//运行结果：1.0,2.0
```

新创建对象，未加以引用

两个引用指向同一个对象

2.6.2 静态域或类域

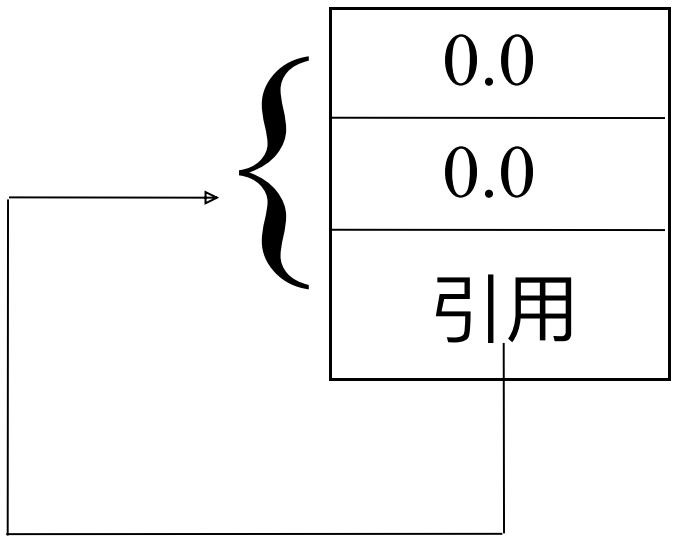
- 同类各对象之间共享的域，也称为类变量。例，工厂生产的每个产品都有唯一的序列号（实例变量），而下一个产品的序列号是类变量。

- 静态域用static来说明，例

```
public class Point {
  public double x,y;
  public static Point origin=new Point();
}
```

类域/静态域

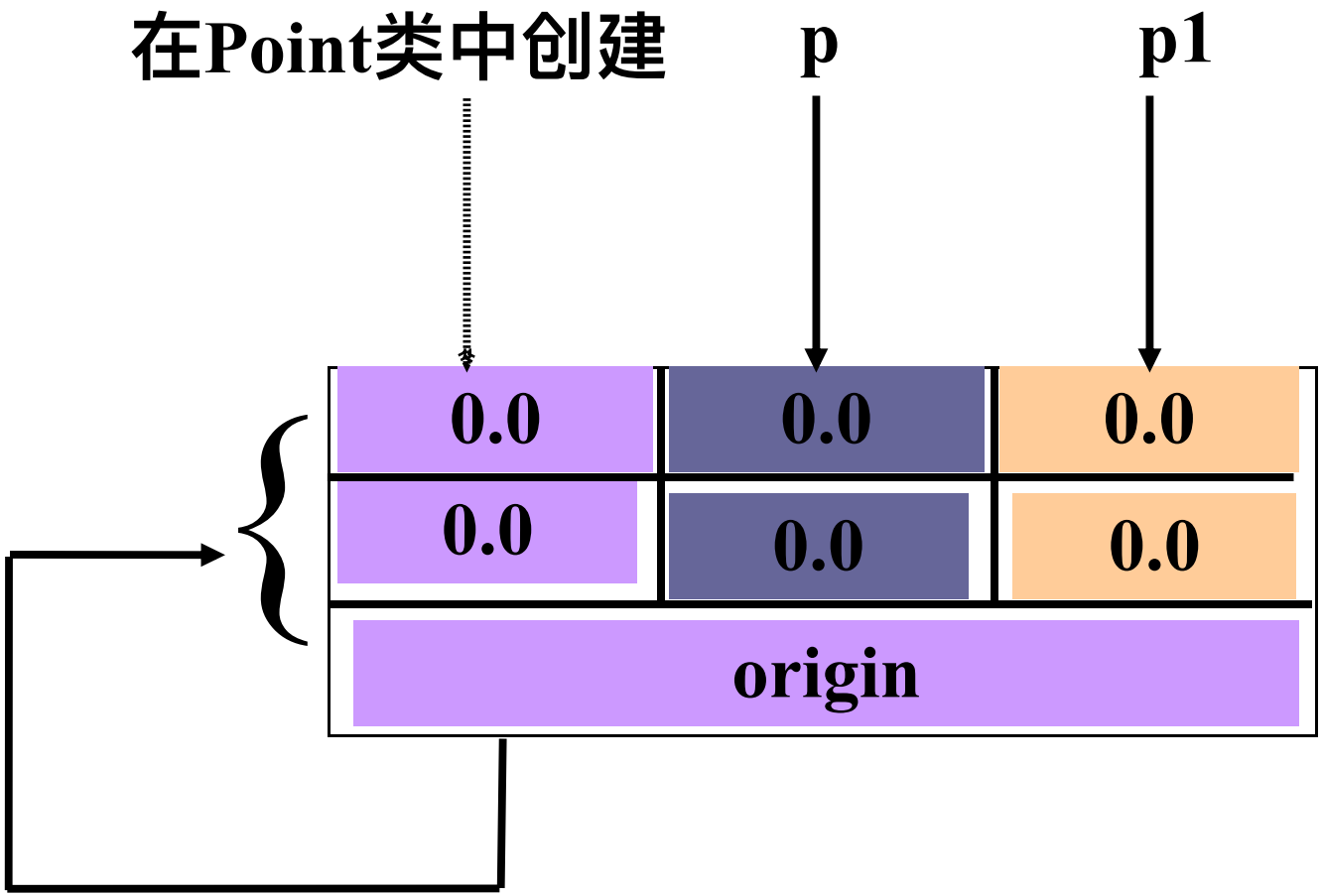
- 不管用Point类创建了多少个对象，甚至一个也没有，静态域总是存在的，且用类名.域名来访问，因此Point.origin指向一个(0.0,0.0)对象。
- 类似于Point.origin，System.out也是一个静态域。
- 下图是在Point类中创建的对象。



- 例

```
System.out.println(Point.origin.x+", "+ Point.origin. y);
//0.0,0.0
Point.origin.x=1; Point.origin.y=2;
System.out.println(Point.origin.x+", "+ Point.origin. y);
//1.0,2.0
Point p=new Point(); Point p1=new Point();
//p和p1指向两个不同的对象
p.x=3;p.y=3;p.origin.x=4;p.origin.y=4; p1.x=5;p1.y=5;
```

```
System.out.println(p.x+", "+p.y); //3.0,3.0
System.out.println(p1.x+", "+p1.y); //5.0,5.0
System.out.println(Point.origin.x+", "+ Point.origin.y);
//4.0,4.0
System.out.println(p.origin.x+", "+ p.origin.y);//4.0,4.0
System.out.println(p1.origin.x+","+p1.origin.y);
//4.0,4.0
System.out.println(Point.origin.origin.x);//4.0
System.out.println(Point.origin. origin.origin.x);//4.0
```


在Point类中创建
p
p1
0.0
0.0
0.0
0.0
0.0
0.0
origin

- 另一例

```
public class Point {
    public double x,y;
    public Point origin=new Point();
}
public class Demo {
    public static void main(String[] args) {
        Point p=new Point();
    }
}
```

static

实例变量

会不断地创建点对象

执行结果如何?

2.6.3 废区收集

- 当一个对象不再有任何引用时，其所占用的资源将被系统在后台回收。

2.7 方法和参数

方法体现了类（对象）的行为，并把私有数据隐藏起来，使其它对象不能直接访问，体现了封装性。

一个设计良好的类隐藏自己的数据，使其仅能被自己类中的方法所改变。

方法可以没有参数，也可以有多个参数。

方法可以有返回值，或者说明为void表示不返回值。

方法的基调(signiture)是方法的名字和参数的数目和类型，也称为原型或型构。

方法的语句用{}括起来，并置于基调之后。

例：在Point类中增加一个clear方法

```
public class Point {
  public double x,y;
  public void clear() {
    x=0;
    y=0;
  }
}
```

访问成员变量

- 在定义方法时，如果要访问方法所属类的域或其它方法均可以直接用其名，无需冠以对象引用。

2.7.1 方法的调用

- 格式：对象引用.方法名（实参表），没有参数的方法，实参表为空。
- 被调用方法所在的对象，称为接受对象或接收者。
- 一个方法可以返回单个值。为了返回多个值，可以在方法中创建对象来保存多个返回值，方法结束前返回这个对象。
- 例：在Point类中再增加一个distance方法，该方法接受另一个Point对象，计算本身对象到这个对象的欧氏距离，返回double值。

```
public class Point {
    …
    public double distance(Point that) {
        double xdiff,ydiff;
        xdiff = x-that.x;
        ydiff = y-that.y;
        return Math.sqrt(xdiff*xdiff+ydiff*ydiff);
    }
}
```

局部变量

实例变量

- double d=lowerLeft.distance(upperRight);

2.7.2 this引用

- 对象对自身的引用，这是一个隐式引用，例：一个等价的clear方法

```
public void clear() {
    this.x=0;
    this.y=0;
}
```

- 通常使用this作为调用其它方法时的一个对象引用实参。
- 使用this可以显式调用当前对象成员，例：move方法

```
public void move(double x,double y) {
    this.x = x;
    this.y = y;
}
```

实例变量

参数

在move方法内，作为参数的x,y隐藏了域

- 一个综合例

```
public class Point {
  public double x,y;
  public void move(double x,double y) {
    x=x; y=y;
  }
  public void move1(double x,double y) {
    this.x=x; this.y=y;
  }
```

参数

参数将实例变量隐藏

达到要求

实例变量

public

```
    static void main(String[] args) {
    Point p=new Point();
    p.move(1,2);
    System.out.println(p.x+", "+p.y); //0.0,0.0
    p.move1(1,2);
    System.out.println(p.x+", "+p.y); //1.0,2.0
    }
}
```

另一例：

```
public class Point {
   …//x,y域及distance方法定义省略
   public double distance1() {
      Point p=new Point();
      p.x=1.0;p.y=1.0;
      return p.distance(this);
   }
}
```

(1,1)　　(0,0)

返回p与当前对象的距离

调用该方法的

```
public class Demo {
  public static void main(String[] args) {
    Point p1=new Point();
    p1.x=3.0;p1.y=1.0;
    System.out.println(p1.distance1());
  }
}
```

结果2.0

2.7.3 静态方法或类方法

- 特指类本身的操作，一般作用于静态域，而不是作用于特定的实例。
- 使用关键词static来说明，用类名.方法名（参数表）来访问。
- 在定义静态方法的语句中不能直接访问本类非静态成员，当调用一个静态方法时，没有特定的对象引用来指明被操作的对象，静态方法完成类的操作，而非静态方法完成对象的操作。
- 一个综合实例

(创建对象后才能访问静态方法)

```
public class Demo {
  static int x;  int y;
  static void addX() {x++;}
  void addY() {y++;x++;}//非静态方法可直接访问静态域
  public static void main(String[] args) {
    System.out.println(x);System.out.println(y); //报错
    Demo p=new Demo();
    System.out.println(p.y);p.addX();
    System.out.println(x);
  }}
```

实例变量未创建对象时是不存在的

创建

静态方法不能直接访问实例变量

2.7.4 可变参数方法

- 在声明方法时，不指定方法的参数个数，使用…

例：public void f (int … x)

- 方法f的参数类型是int，但个数不确定，x是一个参数代表，可以通过下标表示参数列表中的具体参数，x.length等于参数的个数。
- 对于参数类型相同的方法，可以使调用更灵活，既可作为一个数组传递，也可传递一个实参序列。
- 可变形参只能用在最后一个形参位置，而且一个方法中最多只能有一个。

例： class A {

```
    public void f (int … x) {
        int sum=0;
        for (int i=0;i<x.length;i++)
            sum=sum+x[i];
            System.out.println(sum);    }
    public static void main(String[] args)  {
        A a=new A();
        a.f(1,2,3,4,5);
    } }
```

x指向一个数组对象

2.8 数组

- 同一类型变量（引用）的有序集合。

```
class Deck {
  final int DECK_SIZE=52;
  Card[] cards=new Card[DECK_SIZE];
  public void print() {
     for (int i=0;i<cards.length;i++)
        System.out.println(cards[i]);//输出为null
  }
}
```

- 数组创建以后，它的大小就确定了，而且不允许再改变。
- Java把数组处理为对象，length域表示数组对象中元素的数量，数组边界从0到length-1。
- 说明数组变量时维数省略，创建时确定维数：

  int[] ia;

  ia=new int[3];

  可以合写为 int[] ia=new int[3];
- 例题中i的作用域是循环体内。

- 逐元循环(for-each循环):自动遍历数组或者集合的元素，而不是像以前那样手工增加循环的变量，格式如下：

for (类型 变量名称 : 要被遍历的对象) 循环体

逐元循环, JDK1.5以后的新特性。

```
/* 建立一个数组 */
int[] integers = {1, 2, 3, 4};
/* 开始遍历 */
for (int i : integers) {
System.out.println(i);/* 依次输出“1”、“2”、“3”、“4” */
}
```

2.9 字符串对象

- 类型为String，语言直接支持其初始化：

```
System.out.println("Hello,World");
```

编译器实际上创建了一个字符串对象，初始化为指定值，随后将这个对象传给println方法。

- 创建字符串对象，不需要指定其长度，length方法返回字符串中字符数量，即长度，各字符的下标是从0到长度-1。
- 例：

```
class StringDemo {
  static public void main(String[] args) {
    String myName="Petronius";
    myName=myName+ " Arbiter";
    System.out.println("Name=" +myName);
  }
}
```

输出结果：Name=Petronius Arbiter

```
public class BetterStringDemo {
  Static public void main(String[] args) {
    String myName="Petronius";
    String occupation="Reorganization Specialist";
    myName=myName+ " Arbiter";
    myName += " ";
     myName += " ("+occupation+ ") ";
    System.out.println("Name=" +myName);
  }
}// Name=Petronius Arbiter (Reorganization Specialist)
```

字符串对象是只读的，或者说是不可更改的，例

```
str="redwood";
str="oak";
```

str指向了新字符串。

字符串对象的equals方法可以用来比较两个字符串内容是否相同，例

```
if (oneStr.equals(twoStr))
    foundDuplication(oneStr,twoStr);
```

综合例

```
public class StringDemo {
    public static void main(String[] args) {
        String myName="Petronius";
        System.out.println(myName.length());
        System.out.println("Petronius".length());
    }
}
```

结果:

9

9

2.10 类的扩展

- 类的扩展体现了继承性。
- 概念：子类，超类（父类）。
- 子类继承超类的数据和行为，可以增加新的成员，可以改写继承下来的方法。
- Java只支持类的单一继承机制，每个类可以派生多个子类，但每个子类只能有一个直接超类。
- 使用关键词extends，例

```
class Pixel extends Point {
    Color color;
    public void clear() {
        super.clear();
        color=null;
    }
}
```

对超类的引用

- 用子类类型创建的对象既可被引用为子类类型，也可被引用为其超类类型，这也是多态性的表现。

- 任何对超类对象进行处理的代码同样也能作用在子类对象上，因此，如果一个方法有一个Point类型的形参，就可以用Pixel对象作为实参来调用。
- 当一个超类类型的引用指向一个子类类型的具体对象时，不能利用该引用访问超类中未定义的成员，下面是一个综合例

```
class Point {
  public double x=1,y=2;
  public void move(double x, double y) {
     this.x=x;
     this.y=y;
  }
}
```

```
class Pixel extends Point{
  public double z=3;
  public void move(double x, double y) {
     this.x=x+1;
     this.y=y+1;
  }
  public void moveOne(double x, double y) {
     this.x=x+2;
     this.y=y+2;
  }
}
```

子类中新增加成员。

子类中新增加成员。

```
class Demo {
  public static void main(String[] args) {
    Point pointOne=new Pixel();
    System.out.println(pointOne.x);
//      System.out.println(pointOne.x);
    pointOne.move(1,2);
    System.out.println(pointOne.x);
//      pointOne.moveOne(1,2);
  }
}
```

父类，子类

2.10.1 调用超类中的方法

- super.method()被调用时，对象运行系统将从继承的层次结构中查找包含method()方法的超类。例，如果Point没有clear方法，那么会继续查找Point的超类，如此继续，直到找到，并完成调用。
- 对于其它的引用，当调用一个方法时，使用的是对象的实际类型，而不是引用的类型，例

```
Point point=new Pixel();

point.clear();
```

2.10.2 Object类

- 没有显式扩展任何其它类的所有类都隐式地扩展于Object类，这是java的根类，因此Object类型的引用可以指向任何类型的对象。

```
Object oref=new Pixel();
oref="Some String";
```

- Object类的成员将被所有类继承。

2.10.3 类型强制转换

一段代码:

```
String name = "Some String";

Object obj = name; //符合赋值兼容原则

name = obj; //违背赋值兼容原则

name = (String)obj;
```

2.11 接口

- 与类相似，接口也是一种类型，但只给出方法的说明，不提供具体实现。
- 定义接口使用关键词interface，例

```
interface X {
  void methodX();
}
interface Y {
  void methodY();
}
```

- 接口可以扩展，使用关键词extends，且支持多重继承，例

```
interface Z extends X,Y {
  void methodZ();
}
```

- 如果一个类给出了某个接口中所有方法的具体实现，则称该类实现了这个接口，使用关键词implements，一个类可以同时实现多个接口，一个接口可以被多个类实现，例如

```
class SimpleClass implements X,Y {
  public void methodX() {
    System.out.println("methodX");
  }
  public void methodY() {
    System.out.println("methodY");
  }
}
```

```
class SimpleClassOne implements Z {
  public void methodX() {
    System.out.println("methodX");
  }
  public void methodY() {
    System.out.println("methodY");
  }
  public void methodZ() {
    System.out.println("methodZ");
  }
}
```

- 一个类的超类型可以理解为它所继承的类和它所实现的接口，当然也包括这些类和接口的超类型。
- 一个对象的类型不仅是本身类，而且也是其超类型的对象。
- 任何一个对象可以多态地使用，可以作为其任何超类型（超类、超接口）的对象。

```
public class InterfaceDemo {
   public static void main(String[] args) {
      SimpleClass obj=new SimpleClass();
      obj.methodX();obj.methodY();
      SimpleClassOne objOne=new SimpleClassOne();
      objOne.methodX();objOne.methodY();
      objOne.methodZ();
      X objTwo=new SimpleClass();
      objTwo.methodX();//objTwo.methodY();
   }
}
```

一个综合例

```
interface Lookup {
   String find(String name);
}
```

```
class SimpleClass implements Lookup {
  private String[] names={"jane","smith"};
  private String[] values={"female","male"};
  public String find(String name) {
    for (int i=0;i<names.length;i++) {
      if (names[i].equals(name))                    return
 values[i];
    }
    return null;
  }
}
```

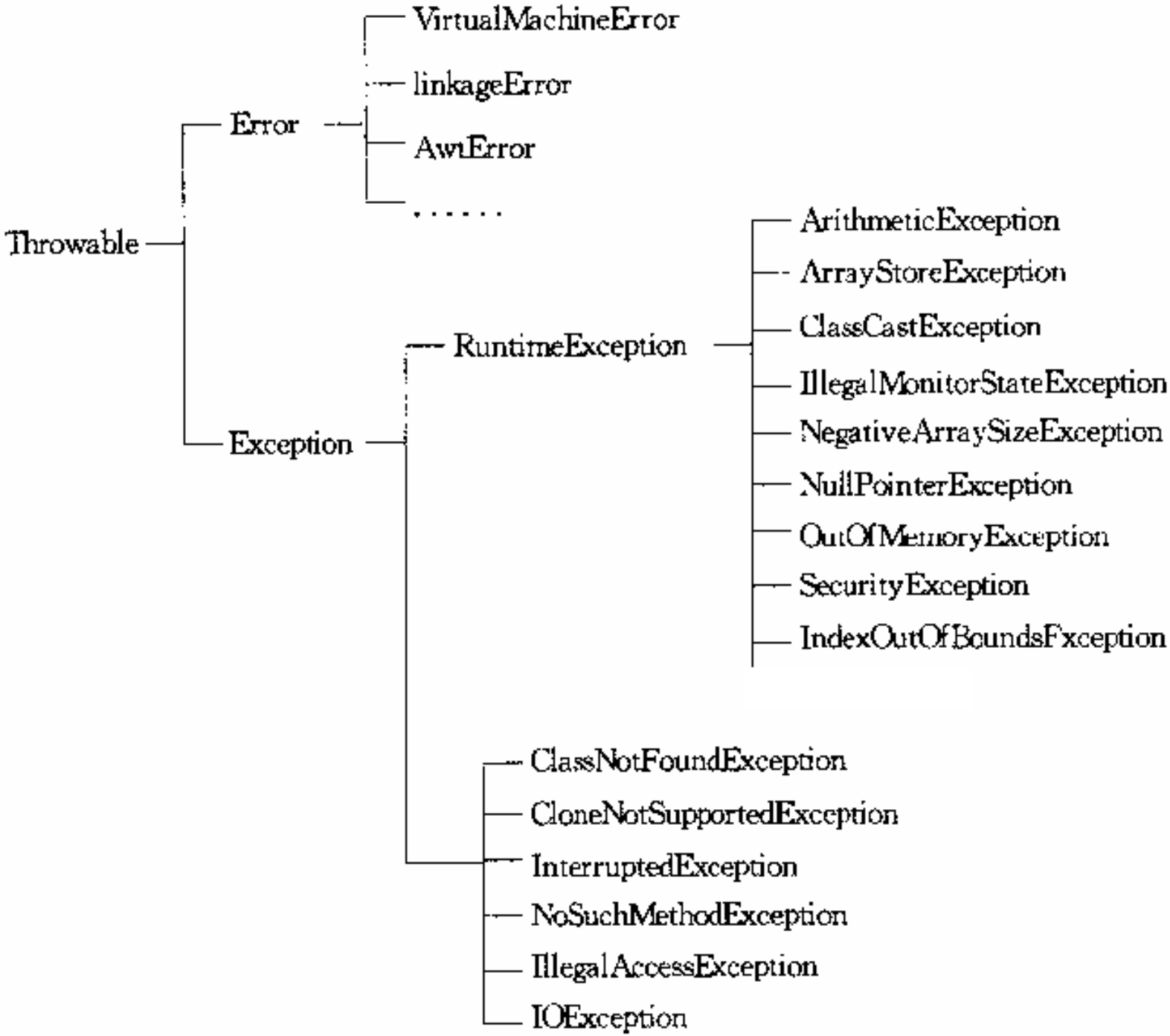
```
public class Demo {
    public static void main(String[] args) {
        String name1="smith";
        Lookup table=new SimpleClass();
        Object value=table.find(name1);
        if (value!=null)
            System.out.println(value);
    }
}
```

2.12 异常和断言

- Java采用异常处理机制来处理程序运行中的错误 。将程序运行中的所有错误都看成一种异常，而且事先编写合适的异常处理程序，通过对语句块的检测， 发现异常后，寻找匹配的异常处理程序进行处理，如果找不到匹配的异常处理程序，则终止程序运行，并显示解释信息。
- Java将程序中发现一个异常并且移交给系统的过程称为引发(throw)一个异常 。
- 根据异常类型寻找匹配的异常处理程序，并由此转入相应的异常处理的过程称为捕获(catch)一个异常。

2.12.1 异常及其分类

- 异常是对象，因此具有类型。Java定义了很多异常类，每个异常类都代表了一种运行错误，类中包含了该运行错误的信息和处理错误的方法。引发一个异常实质上就是创建了相应异常类的一个实例。
- Throwable是java.lang包中一个专门用来处理异常的类。它有两个子类，即Error和Exception，它们分别用来处理两组异常。


Throwable
Error
VirtualMachineError
linkageError
AwtError
.....
Exception
RuntimeException
ArithmeticException
ArrayStoreException
ClassCastException
IllegalMonitorStateException
NegativeArraySizeException
NullPointerException
OutOfMemoryException
SecurityException
IndexOutOfBoundsFxception
ClassNotFoundException
CloneNotSupportedException
InterruptedException
NoSuchMethodException
IllegalAccessException
IOException

- Error用来处理程序运行环境方面的异常，比如，虚拟机错误、装载错误、连接错误资源耗尽，这类异常发生时，除了通知用户并终止程序外基本上无能为力，这种情况是非常罕见的。
- 编程的重点应放在Exception 分支上，Exception下面还有子类，其中一部分子类（RunetimeException及其子类）对应异常产生的原因是编程错误，如程序中除数为0引起的错误、数组下标越界错误等；而任何其它异常的出现原因是好程序遇到了意外情况，比如试图打开一个错误的URL，类找不到、文件没找到等。

当一个方法（构造器）内部出现异常时，可以对其进行处理；也可以不处理，如果是这样，那么该异常会向上传播，方法的调用者仍有机会捕获和处理它。如果异常最终没有被捕获，java解释器会捕获它，并终止执行且打印出错误信息。

- 用户定义的异常应扩展Exception。
- 非检测异常：Error、RunetimeException及其子类。
- 检测异常：其他（包括自定义）,编译器进行检测以确保方法只引发它们自己说明会引发的检测异常。
- 系统定义的异常主要用来处理系统可以预见的较常见的错误，自定义异常针对特定的应用拟补前者的不足，形式上可以通过扩展任何已有的异常类型得到，实际中一般直接扩展于Exception。
- 只有定义了异常类，系统才能识别特定的运行错误，才能及时地控制和处理运行错误，所以定义足够多的异常类是构建一个稳定完美系统的重要基础

2.12.2 异常的引发及捕获处理

- 所有系统定义的异常都可由系统自动引发，例

```
public class TestException {
  public static void main(String[] args) {
     int a=0,b=5;
     System.out.println(b/a);//引发算术异常
  }}
```

Exception in thread "main"
java.lang.ArithmaticException:

/ by zero

所有非抽象异常类在形式上都可用throw语句引发，例

```
public class TestException {
  public static void main(String[] args) {
    throw new InternalError("OK");//Error子类
  }
}
```

Exception in thread "main" java.lang.InternalError: OK

另例

```
public class TestException {
   public static void main(String[] args) {
   throw new ClassCastException();
  //RuntimeException子类
   }
}
```

```
Exception in thread "main"
    java.lang.ClassCastException
```

要捕获异常，需要设置try/catch块，格式如下

try {

可能引发异常的代码

}

catch (异常类型 e) {

异常处理代码

}

只要try块中引发属于catch子句声明类型的异常，那么：

1）程序会跳过try块中的剩余代码；

2）执行catch子句中的代码。

例

```
public class TestException {
  public static void main(String[] args) {
    int a=0,b=5;
    try{System.out.println(b/a);}
    catch (ArithmeticException e) {
      System.out.println("Exception");}
    System.out.println("OK");}}
```

输出： Exception

OK

如果try块中没有引发任何异常，那么程序会跳过catch子句中的代码。例，

```
public class TestException {
  public static void main(String[] args) {
    int a=1,b=5;
    try{System.out.println(b/a);}
    catch (ArithmeticException e) {
      System.out.println("Exception");}
    System.out.println("OK");}}
```

输出： 5

OK

如果try块中引发不属于catch子句声明类型的异常，那么该方法立即退出，异常传播到它的调用者。

```
public class TestException {
  public static void main(String[] args) {
    int a=0,b=5;
    try{System.out.println(b/a);}
    catch (Error e) {
      System.out.println("Exception");}
    System.out.println("OK");}}
```

```
Exception in thread "main"
    java.lang.ArithmaticException: /  by zero
```

- 一个try子句后边可以有多个catch子句，用于捕获try子句中可能引发的多种类型的异常。当try子句中引发一个异常后，依次与每个catch子句中的异常类型进行匹配，找到第一个匹配的catch子句后就执行其中代码，不再与剩下的catch子句进行匹配。如果各catch子句中的异常类型之间有继承关系，则必须把子类型的catch子句放在超类型的catch子句之前，否则编译报错。

- 有时候，捕获异常后，只能做局部应急处理，然后仍需要该方法的调用者进行完全处理，这时可以重新引发异常，例

```
try {
可能引发异常的代码
}
catch (异常类型 e) {
应急处理;
throw e;
}
```

- try/catch块后可以有finally子句，形式如下

```
try {
可能引发异常的代码
}
catch (异常类型 e) {
处理
}
finally {
善后处理
}
```

catch和finally不能同时没有，执行finally子句的三种情况：

try中不引发任何异常，这种情况下，先执行try中所有代码，接着执行finally中代码，然后继续执行try/catch/finally块后面代码；

try中引发一个catch子句能捕获的异常，这种情况下，程序首先执行try中代码直到异常被引发，然后try中剩余代码被跳过，执行匹配的catch子句中代码，最后执行finally中代码；

try中引发一个不能被任何catch子句捕获的异常，这种情况下，程序首先执行try中代码直到异常被引发，然后try中剩余代码被跳过，接着执行finally中代码，并且该异常被传播到方法的调用者

```
public class MyException {
  public static void main(String[] args) {
    int x=1,y=0,z;
    int[] w=new int[2];
    try {
      z=x/y;
      w[2]=2;
      System.out.println("ok");
    }
```

```
catch (ArithmeticException e) {
System.out.println("arithmetic error");
System.out.println(e.toString());
//java.lang.ArithmeticException: / by zero
System.out.println(e.getMessage());//  / by zero
}
catch (IndexOutOfBoundsException e) {
   System.out.println("array error");}
finally {System.out.println("runetime error");}
}}
```

如果一个方法内会引发一个异常（直接或通过调用其它方法间接引发），且在方法内又没有处理，如果该异常是检测异常，则必须在方法定义的throws子句中声明该异常的类型；如果该异常是非检测异常，则可以声明也可以不声明。

```
class MyClass {
  …
  void method() throws ClassNotFoundException, … {
  }
```

说明方法引发的异常类型同说明方法的返回值类型一样重要。

- 方法改写后，其throws子句可以与其超类不同，但不能声明任何不包括在原来方法定义中的异常类型，可以列出比其超类少的异常类型，也可以没有throws子句，意思是不产生检测异常。

2.12.3 综合例

- 自定义异常

```
class UserException extends Exception {
  private int i;
  UserException (int j) {
     i=j;
  }
}
```

```
public class MyException {
  static void dme (int j) throws UserException {
    System.out.println("dme"+j);
    if (j>10)  throw new UserException(j);
    System.out.println("good");
  }
  public static void main(String[] args) {
    try {dme(4);    dme(15);}
    catch (UserException e) {
      System.out.println("Exception is "+e);}}}
```

执行结果:

dme4

good

dme15

Exception is UserException

- 当调用一个在throws子句中列出了一个检测异常的方法时，有三种选择：
  - 捕获该异常并对其进行处理;
  - 捕获该异常并将其映射到一个自己的异常，做法是引发一个在自己throws子句中说明的异常类型;
  - 在自己throws子句中说明该异常，并使该异常直接通过（可能先用finally子句进行清扫）。

```
class MyException extends Exception {
  int x=1;
}
```

```
class Test {
  void aMethod() throws ClassNotFoundException {
    throw new ClassNotFoundException();}
  void bMethod() throws ClassNotFoundException {
    aMethod();}
  void cMethod() throws MyException {
    try {aMethod();}
    catch (ClassNotFoundException e) {
      throw new MyException();
    }
  }
```

```
public static void main(String[] args) throws
    ClassNotFoundException,MyException {
        Test obj=new Test();
        obj.bMethod();
        obj.cMethod();
    }
}
```

2.12.4 断言

- 是JDK1.4以后的新增功能，在调试代码阶段非常有用。
- 一般用于不准备通过捕获异常来处理的错误，如当某个错误发生时，要求程序立即停止执行，在调试代码阶段让断言语句发挥作用，就可以发现一些致命错误。当程序正式运行时可以关闭断言语句，但仍把断言语句保留在源代码中，如果以后又需要调试，可以重新起用断言语句。
- 默认情况下，断言语句是关闭的。
- 可以用-ea或-da在命令行起用或关闭断言语句，例

java –ea mainClass　　java –da mainClass

- 断言语句使用关键词assert来声明，有两种格式：

  assert booleanExpression;

  assert booleanExpression: messageException;

  - 其中， messageException求值是字符串；
  - 如果booleanExpression的值是false时，程序从断言语句处停止执行（第二种情况还输出字符串，提示用户出现的问题）；
  - 如果booleanExpression的值是true时，程序从断言语句处继续执行。

- 例

```
public class AssertExample {
    public static void main(String[] args) {
        double x=-6;
        assert x>0: “负数不能计算平方根”;
        double y=Math.sqrt(x);
        System.out.println(y);
    }
}
```

2.13 包

- 把多个类收集在一起成为一组，称作包(package)
  - 使用包便于组织复杂任务，使不同人的代码分离；
  - 容易保证类名的唯一性。
- 包对应于文件系统的目录，包名与目录名相同。
- 一个物理上的目录在逻辑上包括一个命名包和一个非命名包，命名包的名字与目录名相同。
- 包中不但可以有不同的类，而且可以有子包，对应于子目录。

- 包的名字具有层次结构，用点符号分割各成分。
- 一个类可以使用它所在(逻辑)包中的所有类，也可以使用来自其他包中的public类。
- 使用其他命名包中public类的两种方式
  - 在类名前加上完整的包名，例

```
java.util.Date today=new java.util.Date();
```

  - 可以移入特定的类或一个包中的所有类。

```
import java.util.Date;
import java.util.*;
```

- JDK1.5以前，只能移入类，不能移入成员。例，不能移入 System.out。
- 一条移入语句只能移入一个包中的类，不能同时移入其子包中的类。例，import java.*.*（错）。
- 向包中添加类的方法：把此包的名字放在源文件头部（类定义之前），并使用关键词package，格式：

  package 包名；

  类定义…
- 综合例：

.（当前目录）：

Demo.java, Demo.class

\package1：

OutputPackage1Name.java

OutputPackage1Name.class

\ package2：

OutputPackage2Name.java

OutputPackage2Name.class

以下是各包中的源文件

.\ package1\ package2\OutputPackage2Name.java

```
package package1.package2;
public class OutputPackage2Name {
  public static void outputPackage2Name() {
    System.out.println("package2");
  }
}
```

注意：public类的名字必须与其源文件的名字相同，因此，每个public类独占一个源文件。

.\ package1\ OutputPackage1Name.java

```
package package1;
public class OutputPackage1Name {
   public static void outputPackage1Name() {
      System.out.println("package1");
   }
}
```

在package1这个包中

.\ Demo.java

```
import package1.*;
import package1.package2.*;
//import package1.OutputPackage1Name;
//import package1.package2.OutputPackage2Name;
class Demo {
   static void main(String[] args) {
      OutputPackage1Name.outputPackage1Name();
      OutputPackage2Name.outputPackage2Name();
   }
}
```

另外一种 Demo.java

```
class Demo {
   static void main(String[] args) {
                                                    package1.
    OutputPackage1Name.outputPackage1Name();
                                                    pac          2.
    OutputPackage2Name.outputPackage2Name();
   }
}
```

完整的包前缀。

完整的包前缀。

- 如果没有用包说明，则这个类在非命名包中，非命名包中的类不能在包外被移入和使用。
- 如果使用了包说明，则这个类的字节码文件必须放在与该包名对应的子目录中。
- java.lang包中的类使用前无需用import移入。
- 任何类都在一个特定的包中，要访问某个类，必须能准确地定位它。
- set classpath=…;…; 用来指定类路径，一方面可以作为包移入和包前缀的参考基准路径，另一方面编译器和解释器会在其中的非命名包中查找要访问的类。
- 无论类路径是否包含“.”，编译器总会在当前目录下查找文件。

- jvm在当前路径下搜索类文件的条件
  - 设置类路径，但以“;”结束，甚至类路径中仅仅包括“;”
  - 不设置类路径
- set classpath用来显示设定的类路径。
- set classpath=清除设定的类路径。
- 例

```
D:\lecture\java\2004\example\chap1>path=c:\j2sdk1.4.1_01\bin;
D:\lecture\java\2004\example\chap1>javac HelloWorld.java

D:\lecture\java\2004\example\chap1>java HelloWorld
Hello, World

D:\lecture\java\2004\example\chap1>set classpath
环境变量 classpath 没有定义

D:\lecture\java\2004\example\chap1>set classpath=d:\

D:\lecture\java\2004\example\chap1>java HelloWorld
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorld

D:\lecture\java\2004\example\chap1>set classpath=d:\;.;

D:\lecture\java\2004\example\chap1>java HelloWorld
Hello, World
```

```
D:\lecture\java\2004\example>java HelloWorld
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorld

D:\lecture\java\2004\example>set classpath=d:\lecture\java\2004\example\chap1;

D:\lecture\java\2004\example>java HelloWorld
Hello, World
```

例： chap1\HelloWorld.java

```
public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello, World");
    OK.main(null);}}
```

chap1\next\OK.java

```
public class OK {
  public static void main(String[] args) {
    System.out.println("OK");}}
```

```
D:\lecture\java\2004\example\chap1>path=c:\j2sdk1.4.1_01\bin;
D:\lecture\java\2004\example\chap1>set classpath=

D:\lecture\java\2004\example\chap1>javac HelloWorld.java
HelloWorld.java:5: cannot resolve symbol
symbol  : variable OK
```

```
D:\lecture\java\2004\example\chap1>set classpath=.;.\next;

D:\lecture\java\2004\example\chap1>javac HelloWorld.java

D:\lecture\java\2004\example\chap1>java HelloWorld
Hello, World
OK
```

```
D:\lecture\java\2004\example\chap1>set classpath=

D:\lecture\java\2004\example\chap1>java HelloWorld
Hello, World
Exception in thread "main" java.lang.NoClassDefFoundError: OK
```

例：将HelloWorld放入命名包

```
package chap1;
public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello, World");
  }
}
```

```
D:\lecture\java\2004\example\chap1>set classpath
环境变量 classpath 没有定义

D:\lecture\java\2004\example\chap1>java HelloWorld
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorld
```

```
D:\lecture\java\2004\example\chap1>set classpath=.;

D:\lecture\java\2004\example\chap1>java HelloWorld
Exception in thread "main" java.lang.NoClassDefFoundError: HelloWorld
```

```
D:\lecture\java\2004\example\chap1>set classpath=..;

D:\lecture\java\2004\example\chap1>java chap1.HelloWorld
Hello, World
```

```
D:\lecture\java\2004\example\chap1>cd ..

D:\lecture\java\2004\example>set classpath=.;

D:\lecture\java\2004\example>java chap1.HelloWorld
Hello, World
```

- 静态移入: 移入类中的静态成员，允许它们在没有类前缀时直接按名访问，例

```
import static java.lang.System.*;
public class Test {
  public static void main(String[] args) {
    out.println("Hello World");
  }
}
```

2.14 java的底层结构（自学）

2.15 本章小结

- Java程序由类构成， Java程序设计本质上就是设计类，以及由类组装程序。
- 类有两种成员：域和方法。静态成员定义类的状态和行为，非静态成员定义由类创建的每个对象所具有的状态和行为。
- 同一个类的不同对象具有独立的非静态域，共享静态域。
- 类可以派生子类，java只支持类的单一继承机制。
- 子类继承超类的域和方法，可以增加新的成员，继承下来的方法还可以被改写。

- 与类相似，接口也是一种类型，但只给出方法的说明，不提供具体实现，方法的实现由类提供，Java支持接口的多重继承。
- Java内置了整型、浮点型、布尔型和字符型这四种基本的数据类型，可以用来定义不同类型的变量
- 与基本的数据类型类似，类、接口和数组也可以用来定义不同类型的变量，称为引用，引用既可以指向与引用相同类型或其子类型的具体对象，也可以不指向任何对象（null）。
- 包是组织复杂软件和解决命名冲突的有效方法。
- 字符串和数组处理为对象，length方法返回字符串的长度，length域保存数组的大小，数组创建后，大